

LỜI CAM ĐOAN

Tôi xin cam đoan các kết quả trình bày trong báo cáo đồ án này dựa trên quá trình nghiên cứu và thực nghiệm của riêng tôi, không sao chép bất kỳ kết quả nghiên cứu nào của các tác giả khác. Nội dung của đồ án có tham khảo, sử dụng một số tài liệu, thông tin từ các nguồn tài liệu khoa học, trang web chuyên ngành và các nghiên cứu đã công bố; tất cả đều đã được trích dẫn và liệt kê đầy đủ trong danh mục tài liệu tham khảo.

LỜI CẢM ƠN

Lời đầu tiên, tôi xin bày tỏ lòng biết ơn sâu sắc tới thầy **Nguyễn Việt Nhật** — giảng viên hướng dẫn trực tiếp của tôi. Trong suốt quá trình thực hiện đồ án, thầy đã tận tình định hướng về mặt kỹ thuật, chỉ ra những điểm còn hạn chế và truyền đạt những kinh nghiệm thực tế vô cùng quý giá. Sự hỗ trợ và động viên của thầy là nguồn động lực lớn nhất giúp tôi hoàn thành đề tài này.

Tôi xin trân trọng cảm ơn toàn thể quý thầy cô trong **Khoa Công nghệ Thông tin, Trường Đại học Xây dựng Hà Nội** đã tận tụy truyền đạt kiến thức và tạo mọi điều kiện thuận lợi cho tôi trong suốt quá trình học tập tại trường. Những kiến thức nền tảng được tích lũy qua các môn học là hành trang không thể thiếu để tôi tiếp cận và giải quyết các vấn đề kỹ thuật trong đồ án này.

Cuối cùng, tôi xin gửi lời cảm ơn chân thành nhất tới **gia đình và bạn bè** đã luôn ủng hộ, chia sẻ và đồng hành cùng tôi trong suốt thời gian thực hiện đồ án.

Do kiến thức và kinh nghiệm thực tế còn có những giới hạn nhất định, đồ án này không tránh khỏi những thiếu sót. Tôi rất mong nhận được sự góp ý, nhận xét từ quý thầy cô và các bạn để đề tài có thể được hoàn thiện hơn.

Hà Nội, tháng 8 năm 2026

Sinh viên thực hiện

Nguyễn Ngọc Đức

Mục lục

Danh mục hình vẽ	4
1 MỞ ĐẦU	5
1.1 Lý do chọn đề tài	5
1.2 Mục tiêu nghiên cứu	5
2 CƠ SỞ LÝ THUYẾT VÀ KIẾN TRÚC HỆ THỐNG	6
2.1 Tổng quan về các nền tảng lõi	6
2.2 Phân tách luồng dữ liệu: Metrics và Security Logs	6
2.3 So sánh Prometheus AlertManager và Grafana Alerting	7
2.4 Sơ đồ Kiến trúc và Phân tích Luồng Dữ liệu (Data Flow)	7
3 QUY TRÌNH TRIỂN KHAI THỰC TẾ	9
3.1 Triển khai Nền tảng Giám sát Cơ sở (Prometheus, Grafana & Node Exporter)	9
3.2 Tích hợp và Cấu hình Mạng Wazuh Indexer	14
3.3 Cài đặt Cảm biến An ninh (Wazuh Agent)	15
3.4 Cấu hình Nguồn Dữ liệu (Data Source) trên Grafana	15
3.5 Xây dựng Bảng điều khiển (Dashboards)	15
3.6 Thiết lập Cơ chế Cảnh báo (Alerting) trên hệ sinh thái Wazuh	16
4 TÍCH HỢP ĐIỀU TRA SỰ CỐ	19
4.1 Cơ chế trích xuất dữ liệu tự động (Data Extraction)	19
4.2 Kỹ thuật nhúng mã và Định tuyến (URL Embedding & Routing)	19
4.3 Quy trình phản ứng sự cố thực tế (Automated Incident Response Flow)	19
5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	20
5.1 Kết quả đạt được	20
5.2 Hướng phát triển mở rộng: Định hình Kiến trúc AIOps	20
TÀI LIỆU THAM KHẢO	20

Danh sách hình vẽ

2.1	Sơ đồ luồng dữ liệu và cảnh báo hệ thống	8
3.1	Giao diện kiểm tra trạng thái Targets trên Prometheus	10
3.2	Xác thực luồng dữ liệu đo lường thu thập thành công	11
3.3	Giao diện khởi tạo Contact Point tích hợp Telegram	11
3.4	Khai báo tham số bảo mật Token và Chat ID	12
3.5	Giao diện khởi tạo Rule đo lường tài nguyên	12
3.6	Thiết lập biểu thức toán học đánh giá (Expressions)	13
3.7	Cấu hình điều kiện và ngưỡng kích hoạt (Threshold)	13
3.8	Định dạng nội dung thông báo gửi về hệ thống	14
3.9	Minh họa cảnh báo sự cố trả về nhóm Telegram thành công	14
3.10	Giao diện khởi tạo luồng cảnh báo trên Wazuh	16
3.11	Thiết lập thông số và điều kiện kích hoạt	16
3.12	Cấu hình chi tiết bộ luật an ninh	17
3.13	Thiết lập định tuyến cảnh báo (Routing)	17
3.14	Lưu cấu hình và kích hoạt thành công	18

1. MỞ ĐẦU

1.1 Lý do chọn đề tài

Trong vận hành hệ thống Enterprise truyền thống, người quản trị thường đối mặt với rủi ro "bội thực công cụ"(Tool Fatigue). Khi sự cố xảy ra, họ phải truy cập hàng loạt các bảng điều khiển tách biệt: xem số liệu phần cứng (CPU, RAM) trên Prometheus, kiểm tra log an ninh trên Wazuh/Elasticsearch, hay xem lưu lượng mạng trên Zabbix. Sự phân mảnh này dẫn đến việc xử lý sự cố chậm trễ.

Bên cạnh đó, các công cụ cảnh báo truyền thống thường hoạt động thụ động, thiếu ngữ cảnh và dễ gây ra tình trạng "Spam cảnh báo" khiến người quản trị bỏ lỡ các sự cố nghiêm trọng thực sự.

Do đó, đồ án này đề xuất và triển khai một mô hình hệ thống giám sát thể hệ mới: **Mô hình Trung tâm Điều hành Hợp nhất (Single Pane of Glass)** kết hợp quy trình tối ưu hóa cảnh báo qua Telegram.

1.2 Mục tiêu nghiên cứu

- Phân tách và kết hợp hoàn hảo hai luồng dữ liệu độc lập: **Metrics** (số liệu phần cứng) và **Logs** (sự kiện bảo mật).
- Triển khai **Grafana** làm trung tâm hội tụ đa nguồn (Multi-Datasource).
- Triển khai và tích hợp hệ thống **Wazuh** để phát hiện mã độc và tấn công.
- Thiết lập kênh phản ứng nhanh thông qua **Telegram Bot**.
- Đặt nền móng cho kiến trúc tự động hóa vận hành kết hợp trí tuệ nhân tạo (AIOps).

2. CƠ SỞ LÝ THUYẾT VÀ KIẾN TRÚC HỆ THỐNG

2.1 Tổng quan về các nền tảng lõi

Để xây dựng một hệ thống giám sát toàn diện, đồ án kết hợp sức mạnh của 5 thành phần mã nguồn mở cốt lõi trong lĩnh vực quản trị hệ thống:

- **Prometheus:** Nền tảng giám sát hệ thống dựa trên cơ sở dữ liệu chuỗi thời gian (Time-Series Database - TSDB). Khác với các cơ sở dữ liệu quan hệ truyền thống, Prometheus được thiết kế chuyên biệt để ghi nhận khối lượng lớn các điểm dữ liệu đo lường (metrics) ở tốc độ cao, thông qua cơ chế chủ động truy vấn (Pull Mechanism).
- **Node Exporter:** Đóng vai trò là tác tử (Agent) đo lường phần cứng. Chức năng chính của Node Exporter là trích xuất các thông số từ lõi hệ điều hành (CPU, RAM, Disk, Network), sau đó "phiên dịch"(export) chúng sang định dạng tiêu chuẩn (Prometheus exposition format) và phơi bày qua giao thức HTTP để máy chủ Prometheus thu thập.
- **Wazuh SIEM:** Hệ thống quản lý thông tin và sự kiện an toàn bảo mật. Wazuh hoạt động theo mô hình Client-Server, thu thập nhật ký hệ thống (Logs) từ Agent, đối soát liên tục với bộ quy tắc (Ruleset) chuẩn MITRE ATT&CK nhằm phát hiện và định danh các hành vi tấn công mạng.
- **OpenSearch (Wazuh Indexer):** Công cụ phân tích và tìm kiếm dữ liệu phân tán (được phát triển từ Elasticsearch). Trong đồ án này, OpenSearch đóng vai trò là cơ sở dữ liệu cốt lõi của Wazuh, chuyên xử lý và đánh chỉ mục (Indexing) các cảnh báo an ninh, hỗ trợ truy vấn tìm kiếm toàn văn bản (Full-text search) với hiệu năng cao.
- **Grafana:** Nền tảng trực quan hóa dữ liệu, đóng vai trò "Màn hình điều khiển hợp nhất"(Single Pane of Glass). Grafana không trực tiếp lưu trữ dữ liệu; thay vào đó, hệ thống thực hiện các truy vấn đồng thời tới nhiều nguồn dữ liệu (Prometheus, OpenSearch) nhằm tổng hợp biểu đồ và thiết lập cơ sở kích hoạt cảnh báo.

2.2 Phân tách luồng dữ liệu: Metrics và Security Logs

Một kiến trúc giám sát tiêu chuẩn Enterprise yêu cầu sự tách biệt rõ ràng giữa hai luồng dữ liệu đặc thù:

Tiêu chí	Luồng Metrics (Prometheus)	Luồng Logs & Events (Wazuh / OpenSearch)
Bản chất	Dữ liệu định lượng (Numbers) biến thiên theo thời gian.	Chuỗi văn bản chi tiết (Text & Strings) lưu vết sự kiện.
Ví dụ	RAM = 80%, CPU = 90%	"Invalid user hieunn from 192.168.68.181 port 36958"
Lưu trữ	Prometheus TSDB (Cấu trúc block tối ưu nén dữ liệu).	Wazuh Indexer (Tối ưu đánh chỉ mục và tìm kiếm văn bản).

Điểm nhấn lý thuyết: Cơ sở dữ liệu của Prometheus được thiết kế chuyên biệt cho kiểu dữ liệu số (Float/Integer). Việc lưu trữ dữ liệu an toàn thông tin (chứa IP, Username, chuỗi lệnh) bắt buộc phải được đảm

nhiệm bởi một kho dữ liệu tài liệu (Document-oriented Database) như **Wazuh Indexer**, nhằm bảo toàn nguyên vẹn ngữ cảnh của sự kiện.

2.3 So sánh Prometheus AlertManager và Grafana Alerting

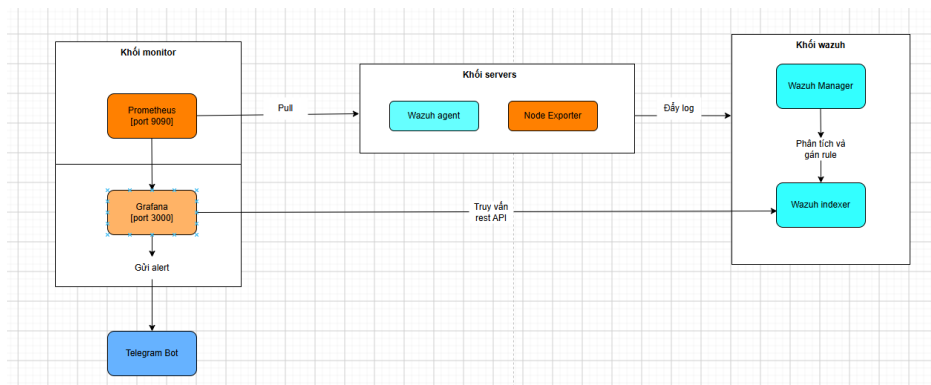
Mặc dù hệ sinh thái Prometheus cung cấp sẵn AlertManager, đồ án lựa chọn **Grafana Alerting** làm cơ chế báo động trung tâm dựa trên các ưu điểm sau:

1. **Khả năng Đa nguồn (Multi-Datasource):** AlertManager bị giới hạn trong phạm vi dữ liệu của Prometheus. Ngược lại, Grafana hỗ trợ truy xuất và tổng hợp cảnh báo từ đa dạng nguồn dữ liệu, bao gồm cả nhật ký an ninh từ Wazuh.
2. **Khuôn mẫu tin nhắn (Message Templates):** Grafana cung cấp công cụ tùy biến nội dung thông báo thông qua ngôn ngữ đánh dấu HTML/Go Template, tối ưu hóa giao diện hiển thị trên Telegram.
3. **Phép toán kết hợp:** Cho phép kết hợp và tính toán biểu thức (Expressions) từ nhiều nguồn dữ liệu khác nhau trước khi ra quyết định kích hoạt cảnh báo.

2.4 Sơ đồ Kiến trúc và Phân tích Luồng Dữ liệu (Data Flow)

Kiến trúc hệ thống được phân chia thành 4 khối chức năng, hoạt động theo luồng dữ liệu như sau:

1. **Khối Client (Máy chủ mục tiêu):** Nơi triển khai các tác tử thu thập dữ liệu bao gồm Node Exporter (giám sát tài nguyên phần cứng) và Wazuh Agent (giám sát an toàn thông tin).
2. **Khối Wazuh SIEM (Xử lý Log):**
 - *Wazuh Agent* phát hiện các sự kiện hệ thống và đẩy (Push) dữ liệu thô về *Wazuh Manager*.
 - *Wazuh Manager* đóng vai trò xử lý trung tâm, giải mã log, đối chiếu quy tắc bảo mật và gán nhãn Rule ID cùng mức độ rủi ro (Level).
 - Dữ liệu sự kiện sau đó được dịch vụ Filebeat chuyển tiếp tới *Wazuh Indexer (OpenSearch)* để lưu trữ và đánh chỉ mục.
3. **Khối Master (Giám sát trung tâm):**
 - *Prometheus* duy trì chu kỳ định kỳ (Pull) để truy xuất dữ liệu phần cứng từ các Node Exporter.
 - *Grafana* hoạt động như giao diện trung tâm, sử dụng PromQL để truy vấn dữ liệu từ Prometheus và giao thức REST API (cổng 9200) để truy vấn nhật ký an ninh từ Wazuh Indexer.
4. **Khối Cảnh báo (Alerting):** Grafana đánh giá các biểu thức giám sát; khi dữ liệu vượt ngưỡng (Threshold), hệ thống sẽ gửi yêu cầu POST (HTTPS) tới Telegram Bot API để phát đi thông báo.



Hình 2.1 – Sơ đồ luồng dữ liệu và cảnh báo hệ thống

3. QUY TRÌNH TRIỂN KHAI THỰC TẾ

3.1 Triển khai Nền tảng Giám sát Cơ sở (Prometheus, Grafana & Node Exporter)

Để đảm bảo tính linh hoạt và dễ dàng cô lập môi trường, hệ thống giám sát trung tâm được triển khai hoàn toàn trên nền tảng Container (Docker). Dưới đây là các bước cấu hình chi tiết để khởi chạy:

Bước 1: Cấu hình hệ thống Prometheus và Grafana trên máy chủ Master

1. Tạo thư mục chứa dữ liệu và tệp cấu hình:

```
1 sudo mkdir -p /opt/monitoring
2 cd /opt/monitoring
```

2. Khởi tạo tệp cấu hình prometheus.yml để định tuyến mục tiêu thu thập dữ liệu:

```
1 global:
2   scrape_interval: 15s
3 scrape_configs:
4   - job_name: 'prometheus'
5     static_configs:
6       - targets: ['localhost:9090']
```

3. Tạo tệp docker-compose.yml định nghĩa cụm vi dịch vụ:

```
1 version: '3.8'
2 services:
3   prometheus:
4     image: prom/prometheus:latest
5     ports:
6       - "9090:9090"
7     volumes:
8       - ./prometheus.yml:/etc/prometheus/prometheus.yml
9     restart: unless-stopped
10
11   grafana:
12     image: grafana/grafana:latest
13     ports:
14       - "3000:3000"
15     restart: unless-stopped
```

4. Kích hoạt và chạy ngầm cụm dịch vụ:

```
1 sudo docker compose up -d
```

Bước 2: Cài đặt công cụ đo lường Node Exporter trên máy con (Clients)

Để công cụ tự động khởi động cùng hệ điều hành, Node Exporter được cấu hình chuẩn theo dạng Systemd Service. Quản trị viên chỉ cần sao chép và chạy toàn bộ khối lệnh sau:

```
1 # T i v g i i n n p h n m m
2 cd /tmp
3 wget https://github.com/prometheus/node_exporter/releases/download/v1.8.2/
   node_exporter-1.8.2.linux-amd64.tar.gz
4 tar -xvf node_exporter-1.8.2.linux-amd64.tar.gz
5 sudo mv node_exporter-1.8.2.linux-amd64/node_exporter /usr/local/bin/
```

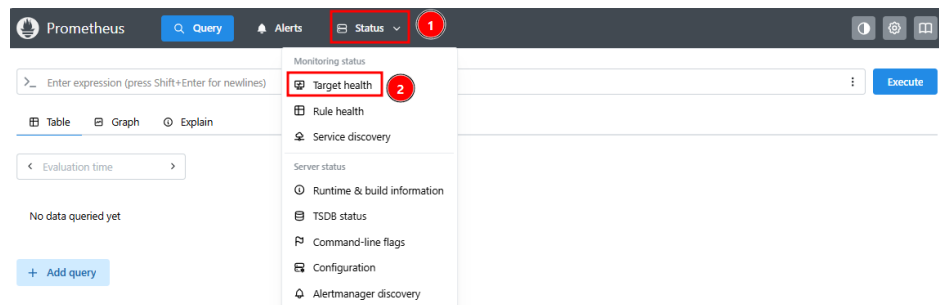
```

6
7 # T o t i k h o n h   t h n g   c   l p           t n g   t n h   b o   m t
8 sudo useradd -rs /bin/false node_exporter
9
10 # T o   f i l e   c u   h   n h   d   c h   v   S y s t e m d   t   k   h   i   n g
11 sudo bash -c 'cat <<EOF > /etc/systemd/system/node_exporter.service
12 [Unit]
13 Description=Node Exporter
14 After=network.target
15
16 [Service]
17 User=node_exporter
18 Group=node_exporter
19 Type=simple
20 ExecStart=/usr/local/bin/node_exporter
21
22 [Install]
23 WantedBy=multi-user.target
24 EOF'
25
26 # K   c h   h   o   t   v   c   h   y   d   c h   v
27 sudo systemctl daemon-reload
28 sudo systemctl enable node_exporter
29 sudo systemctl start node_exporter

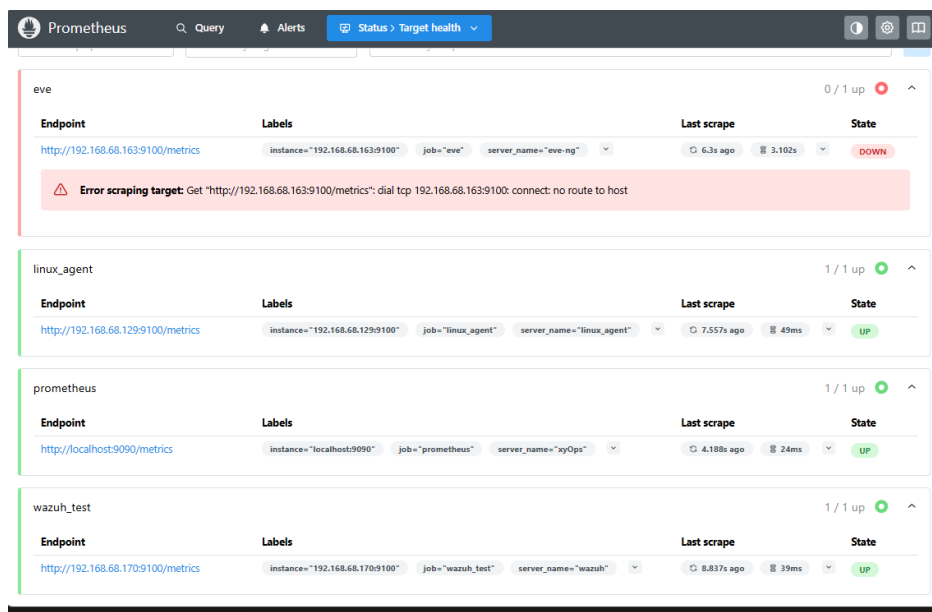
```

Bước 3: Xác thực luồng dữ liệu Metrics trên Prometheus

Sau khi hoàn tất tiến trình cài đặt Node Exporter, quản trị viên truy cập vào giao diện web của Prometheus để kiểm tra trạng thái hệ thống mục tiêu (Targets) nhằm đảm bảo dữ liệu phần cứng (Metrics) đang được thu thập ổn định:



Hình 3.1 – Giao diện kiểm tra trạng thái Targets trên Prometheus



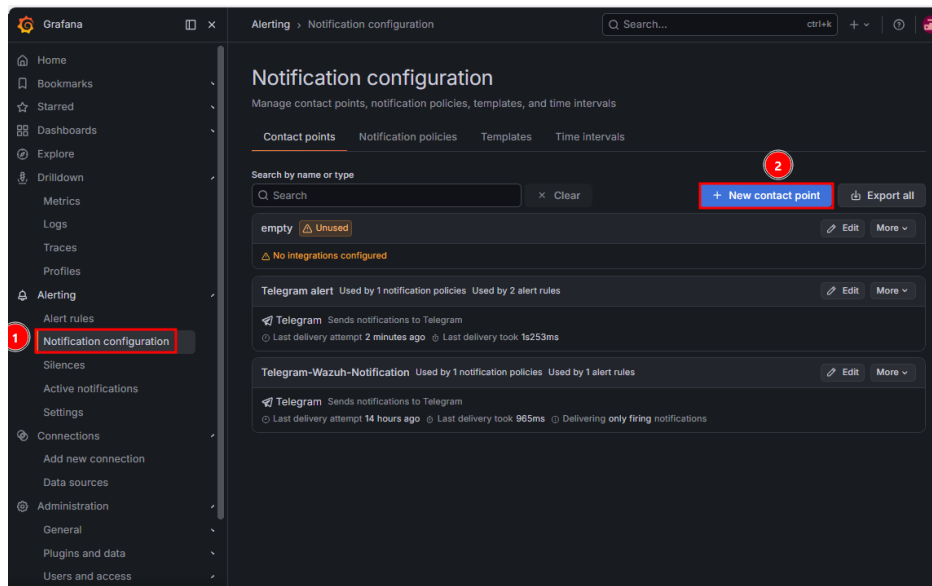
Hình 3.2 – Xác thực luồng dữ liệu đo lường thu thập thành công

Bước 4: Thiết lập Cơ chế Cảnh báo Đo lường (Grafana Alerting)

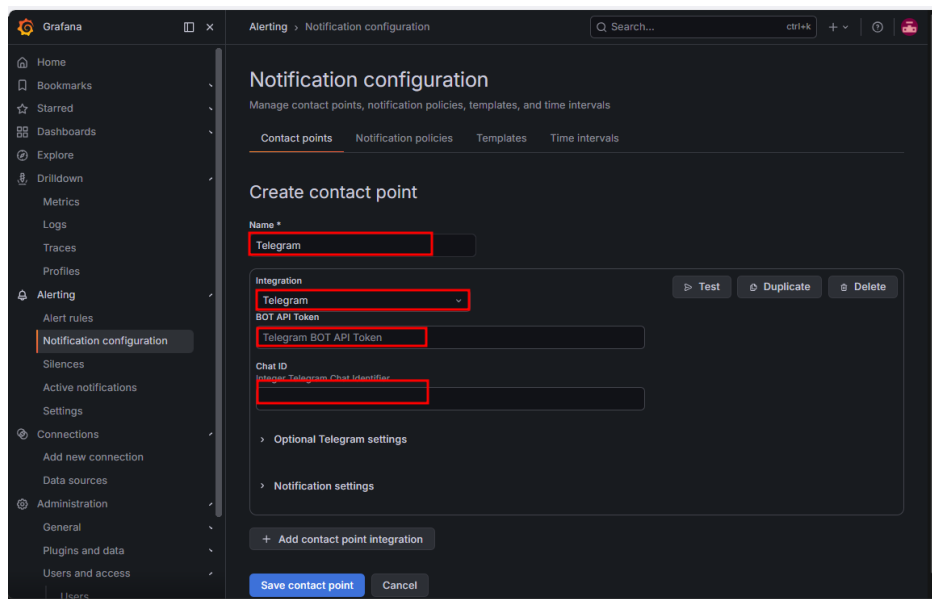
Hệ thống giám sát được tích hợp khả năng phản ứng tức thời thông qua việc định tuyến cảnh báo từ Grafana về Telegram:

1. Khai báo kênh tiếp nhận (Contact Points)

Quá trình kết nối tới Telegram yêu cầu xác thực thông qua hai tham số bảo mật: BOT API Token và Chat ID.



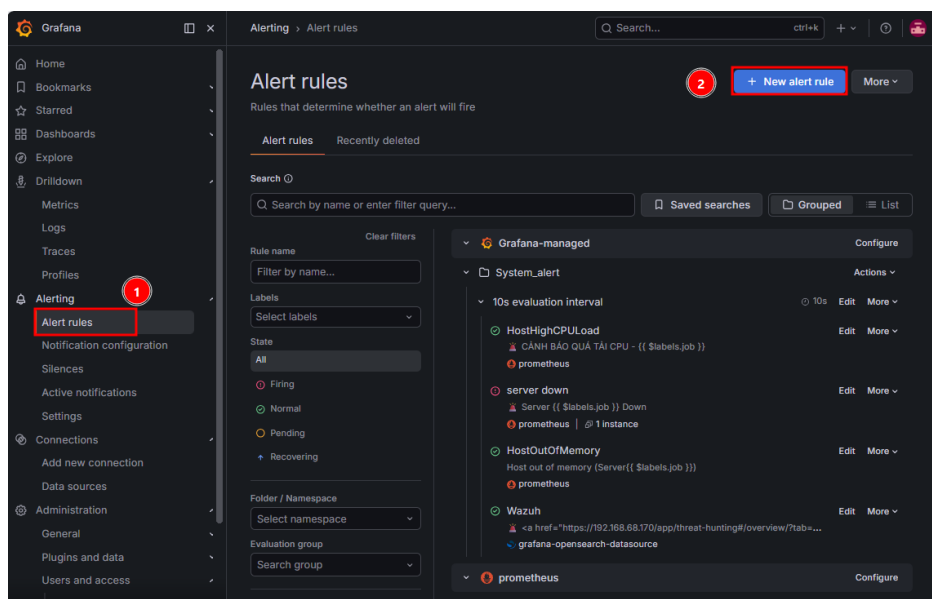
Hình 3.3 – Giao diện khởi tạo Contact Point tích hợp Telegram



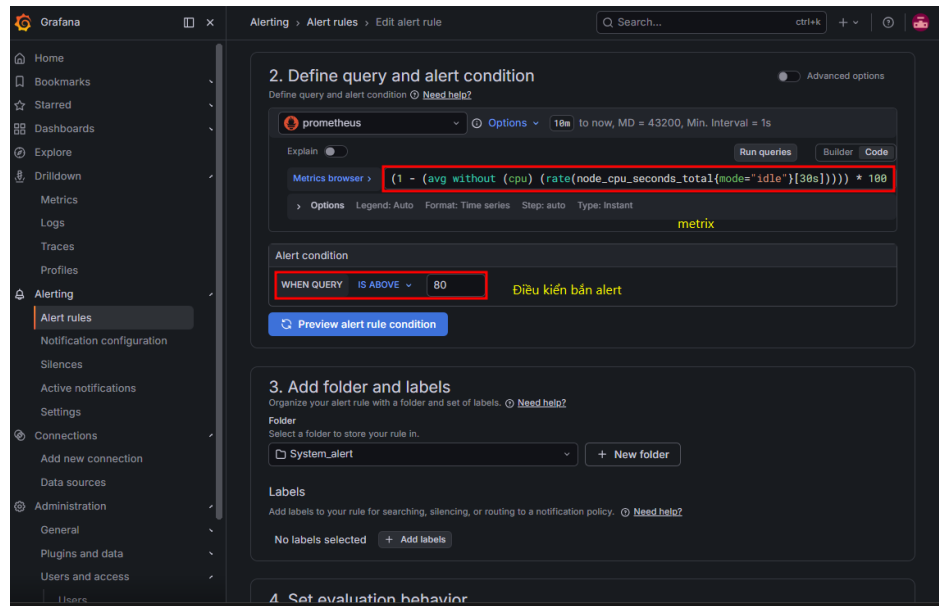
Hình 3.4 – Khai báo tham số bảo mật Token và Chat ID

2. Khai báo Luật cảnh báo (Alert Rules)

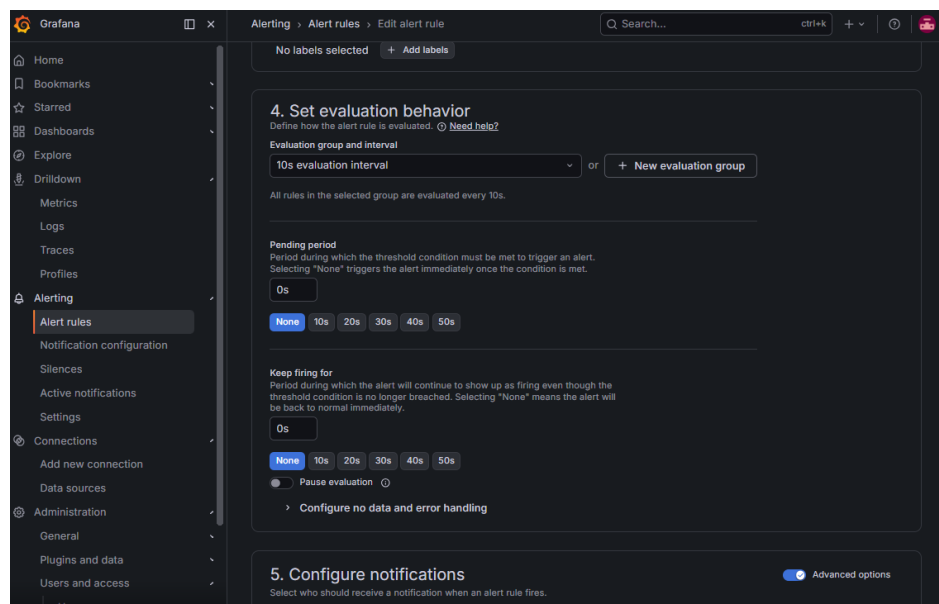
Luật cảnh báo được cấu trúc dựa trên PromQL nhằm đánh giá liên tục các chỉ số tài nguyên (ví dụ: cảnh báo khi CPU hoặc RAM bị quá tải).



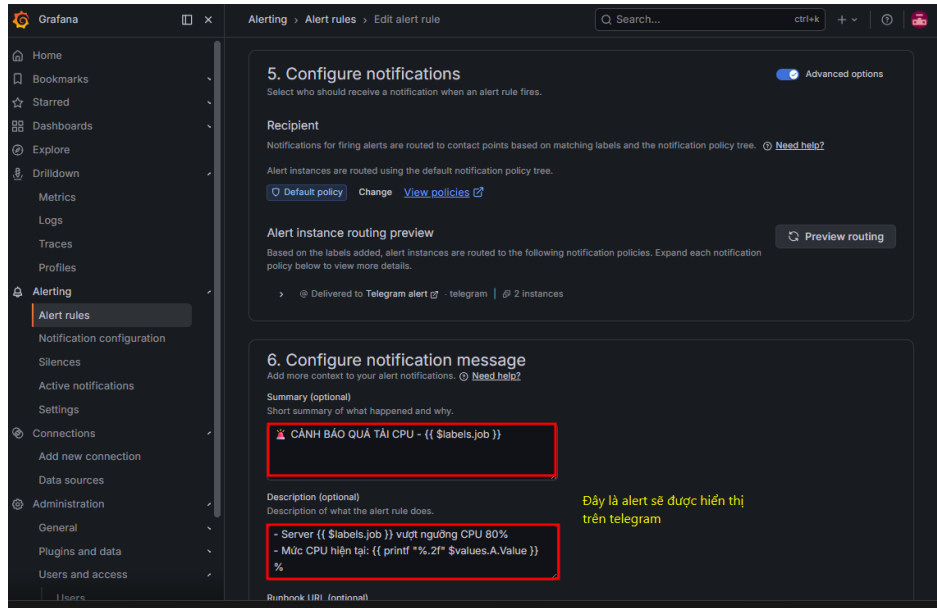
Hình 3.5 – Giao diện khởi tạo Rule đo lường tài nguyên



Hình 3.6 – Thiết lập biểu thức toán học đánh giá (Expressions)



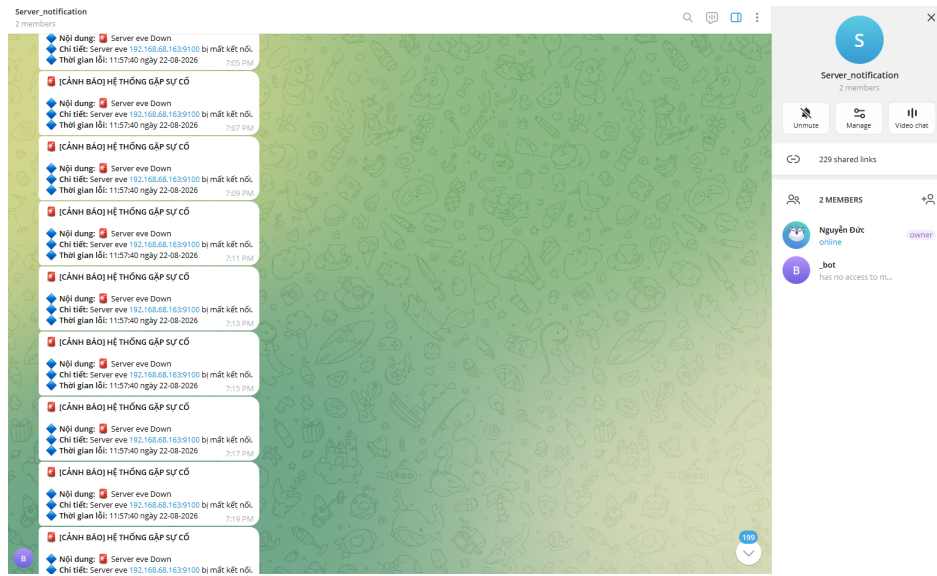
Hình 3.7 – Cấu hình điều kiện và ngưỡng kích hoạt (Threshold)



Hình 3.8 – Định dạng nội dung thông báo gửi về hệ thống

3. Xác thực kết quả tích hợp

Sau khi luồng cấu hình hoàn tất, hệ thống sẽ tự động phân tích và đẩy tín hiệu báo động về ứng dụng Telegram của người quản trị mỗi khi phát sinh vi phạm tài nguyên.



Hình 3.9 – Minh họa cảnh báo sự cố trả về nhóm Telegram thành công

3.2 Tích hợp và Cấu hình Mạng Wazuh Indexer

Theo thiết kế mặc định vì lý do bảo mật, Wazuh Indexer chỉ chấp nhận kết nối cục bộ (Loopback 127.0.0.1). Để Grafana có thể truy xuất dữ liệu từ bên ngoài, cấu hình mạng cần được can thiệp sâu:

1. **Chỉnh sửa cấu hình lỗi:** Mở tệp `/etc/wazuh-indexer/opensearch.yml` và thay đổi tham số bind IP `network.host` thành `0.0.0.0`. Lệnh can thiệp:

```
sudo sed -i 's/network.host: "127.0.0.1"/network.host: "0.0.0.0"/' /etc/wazuh-
```

```
indexer/opensearch.yml
```

2. **Khởi tạo lại cấu trúc bảo mật:** Việc đổi IP sẽ phá vỡ xác thực nội bộ của Cluster. Quản trị viên tiến hành chạy tập lệnh `indexer-security-init.sh` kèm theo chứng thư số (`admin.pem`) để nạp lại cấu hình bảo mật mới vào bộ nhớ mà không làm gián đoạn dịch vụ:

```
1 sudo /usr/share/wazuh-indexer/bin/indexer-security-init.sh
```

3.3 Cài đặt Cảm biến An ninh (Wazuh Agent)

Cảm biến an ninh được triển khai đồng loạt trên các máy chủ mục tiêu thông qua gói cài đặt `.deb`.

- Trong quá trình cài đặt, biến môi trường `WAZUH_MANAGER` được khai báo trở trực tiếp về địa chỉ IP của máy chủ xử lý trung tâm (Wazuh Server). Lệnh cài đặt tự động hóa:

```
1 wget -q https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4
  .12.0-1_amd64.deb -O /tmp/wazuh-agent.deb
2 sudo WAZUH_MANAGER='192.168.68.170' WAZUH_AGENT_NAME='xyops' dpkg -i /tmp/wazuh-
  agent.deb
```

- Sau khi kích hoạt dịch vụ, trạng thái kết nối được xác thực bằng công cụ quản lý nội bộ của hệ thống:

```
1 sudo /var/ossec/bin/agent_control -l
```

3.4 Cấu hình Nguồn Dữ liệu (Data Source) trên Grafana

Để Grafana có thể đọc hiểu định dạng của Wazuh Indexer (OpenSearch), cần thực hiện hai bước tích hợp:

- **Cài đặt Plugin:** Triển khai gói mở rộng `grafana-opensearch-datasource` trực tiếp vào vùng chứa (Container) của Grafana thông qua công cụ `grafana-cli`:

```
1 sudo docker exec grafana grafana cli plugins install grafana-opensearch-
  datasource
```

- **Thiết lập kết nối:** Khai báo Endpoint API tới cổng 9200. Chế độ định tuyến được thiết lập là **Proxy** nhằm tránh lỗi chính sách nguồn gốc chéo (CORS). Thông số xác thực sử dụng tài khoản `admin` nội bộ, kèm theo thiết lập bỏ qua kiểm duyệt chứng thư số tự ký (TLS Skip Verify). Dữ liệu được trỏ đích xác vào Index Pattern `wazuh-alerts-*`.

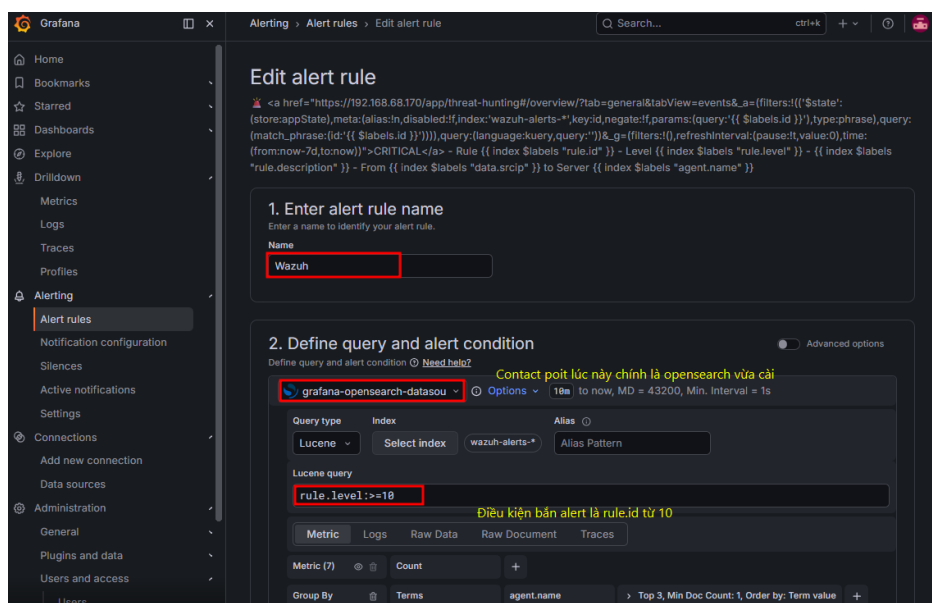
3.5 Xây dựng Bảng điều khiển (Dashboards)

Sau khi hội tụ thành công các nguồn dữ liệu, các bảng điều khiển (Dashboard) được thiết kế để cung cấp bức tranh toàn cảnh về hệ thống.

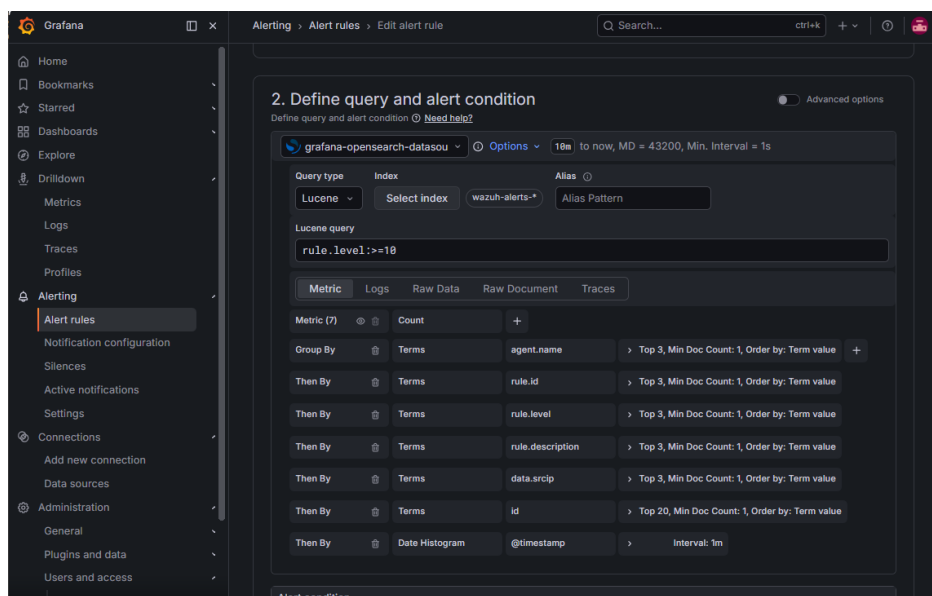
- Biểu đồ Metrics phản ánh mức độ tiêu thụ tài nguyên phần cứng (CPU, RAM, Disk I/O) theo thời gian thực.
- Bảng Log an ninh thống kê chi tiết các sự kiện tấn công bị Wazuh ngăn chặn, phân loại theo mức độ nghiêm trọng (Level) và Rule ID.

3.6 Thiết lập Cơ chế Cảnh báo (Alerting) trên hệ sinh thái Wazuh

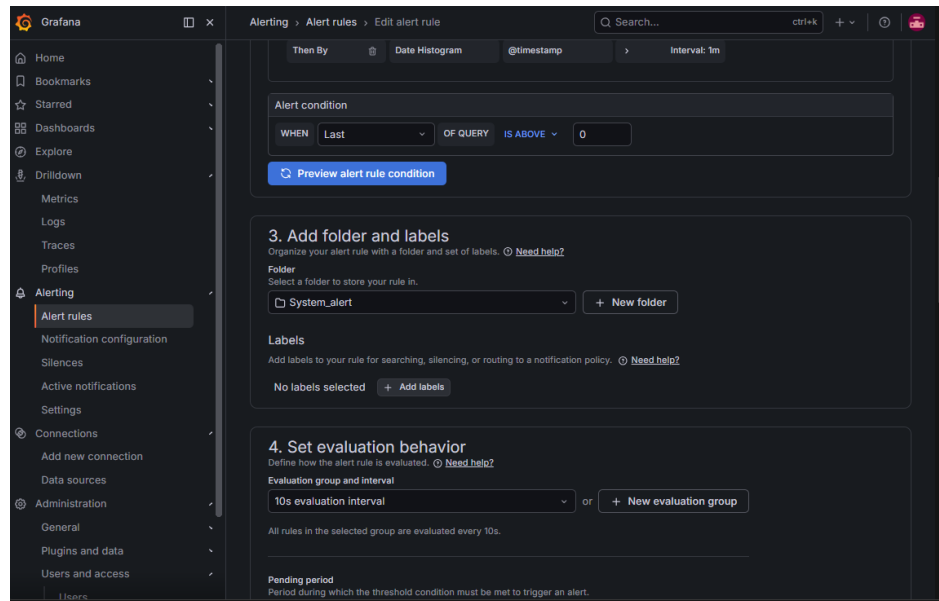
Quy trình tinh chỉnh và kích hoạt các luồng cảnh báo an ninh được thao tác trực tiếp trên giao diện quản trị. Chuỗi thao tác thực tế được minh họa chi tiết dưới đây:



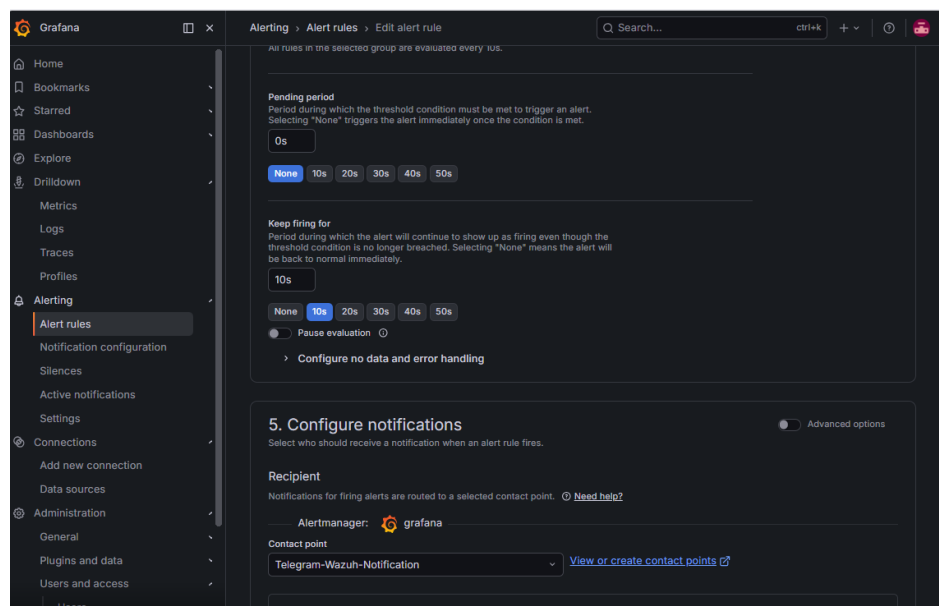
Hình 3.10 – Giao diện khởi tạo luồng cảnh báo trên Wazuh



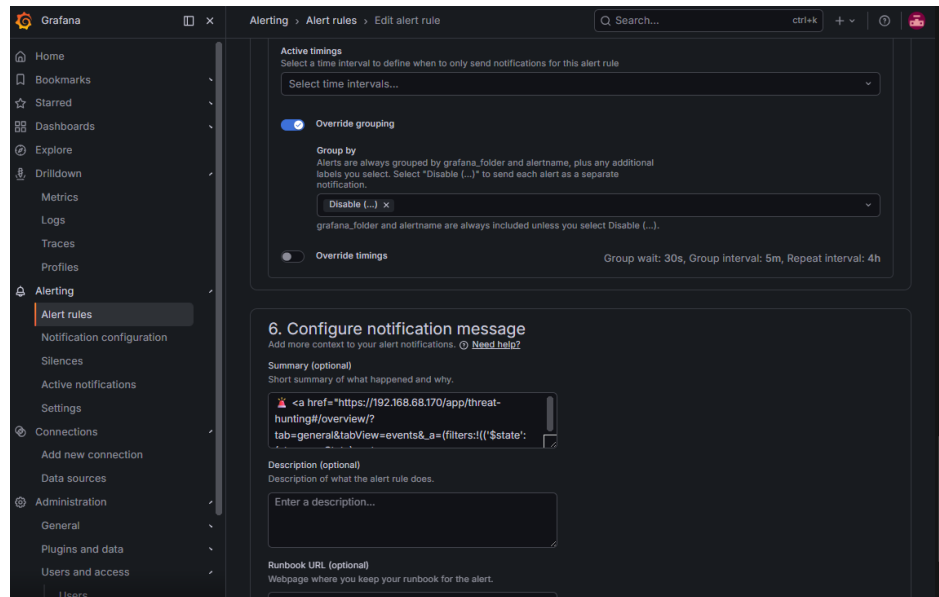
Hình 3.11 – Thiết lập thông số và điều kiện kích hoạt



Hình 3.12 – Cấu hình chi tiết bộ luật an ninh



Hình 3.13 – Thiết lập định tuyến cảnh báo (Routing)



Hình 3.14 – Lưu cấu hình và kích hoạt thành công

4. TÍCH HỢP ĐIỀU TRA SỰ CỐ

Một trong những điểm nhấn kỹ thuật quan trọng nhất của hệ thống là khả năng liên kết sâu (Deep Linking) giữa giao diện cảnh báo (Telegram) và giao diện truy vết (Wazuh Threat Hunting), giúp thu hẹp tối đa thời gian phản ứng sự cố (MTTR - Mean Time To Respond).

4.1 Cơ chế trích xuất dữ liệu tự động (Data Extraction)

Khi Wazuh phát hiện sự cố, hệ thống không chỉ phát cảnh báo chung chung mà còn đóng gói toàn bộ ngữ cảnh tấn công (IP nguồn, tài khoản bị xâm phạm, mã lỗi) thành một tài liệu JSON.

Khi Grafana truy vấn Wazuh Indexer thông qua giao thức REST API, hệ thống Grafana Alerting sẽ tự động phân tích (parse) cấu trúc JSON này và trích xuất các giá trị quan trọng thành các Nhãn động (Dynamic Labels). Ví dụ điển hình: IP thực của kẻ tấn công sẽ được hệ thống bóc tách và gán vào biến số `{{ index .Labels "data.srcip" }}`.

4.2 Kỹ thuật nhúng mã và Định tuyến (URL Embedding & Routing)

Để biến tin nhắn Telegram từ một đoạn văn bản thuần túy thành một công cụ điều tra tương tác, đồ án đã tiến hành can thiệp vào cấu trúc khuôn mẫu (Message Template) của Grafana thông qua ngôn ngữ Go Template.

Cụ thể, một đường link định tuyến URL được sinh tự động và nhúng trực tiếp vào nút bấm **"CRITICAL"**. Đường link này tuân thủ cấu trúc mã hóa RISON của OpenSearch Dashboards, cho phép truyền thẳng câu lệnh truy vấn (KQL - Kuery) vào thanh tìm kiếm của trình duyệt:

```
https://<WAZUH_IP>/app/discover#/?_a=(query:(language:kuery,query:'data.srcip:
"{{ index .Labels "data.srcip" }}"'))
```

4.3 Quy trình phản ứng sự cố thực tế (Automated Incident Response Flow)

Nhờ sự kết hợp của hai kỹ thuật trên, luồng vận hành SOC thực tế được tối ưu hóa ở mức cao nhất:

1. Grafana phát hiện cuộc tấn công (Ví dụ: SSH Brute-force) và bắn tin nhắn cảnh báo tới nhóm điều hành trên Telegram.
2. Quản trị viên (SOC Analyst) nhấn vào nút **"CRITICAL"** đính kèm phía dưới tin nhắn.
3. Trình duyệt lập tức chuyển hướng và đăng nhập thẳng vào giao diện OpenSearch Dashboards của Wazuh SIEM.
4. Dựa vào bộ mã RISON, thanh tìm kiếm KQL của Wazuh tự động điền sẵn bộ lọc truy vấn bằng chính IP của kẻ tấn công (Ví dụ: `data.srcip: "192.168.68.181"`).
5. Quản trị viên ngay lập tức quan sát được toàn bộ chuỗi hành vi của tác nhân độc hại trên một màn hình duy nhất, lược bỏ hoàn toàn các bước tra cứu và trích xuất log thủ công tốn thời gian.

5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết quả đạt được

Đồ án đã triển khai thành công kiến trúc giám sát và cảnh báo an toàn thông tin tập trung, đáp ứng các tiêu chuẩn vận hành hệ thống cấp độ doanh nghiệp (Enterprise). Những đóng góp chính của đề tài bao gồm:

- **Tối ưu hóa quy trình vận hành:** Khắc phục triệt để tình trạng phân mảnh công cụ giám sát bằng việc thiết lập thành công mô hình "Single Pane of Glass" trên nền tảng Grafana.
- **Quy hoạch luồng dữ liệu thông minh:** Phân tách và xử lý hiệu quả hai luồng dữ liệu đặc thù là Metrics (qua Prometheus) và Security Logs (qua hệ sinh thái Wazuh).
- **Nâng cao năng lực phản ứng sự cố (Incident Response):** Triển khai thành công kỹ thuật liên kết sâu (Deep Linking) thông qua mã hóa RISON, giúp tự động hóa quy trình truy vết tác nhân đe dọa trực tiếp từ nền tảng Telegram.

5.2 Hướng phát triển mở rộng: Định hình Kiến trúc AIOps

Dựa trên nền tảng dữ liệu đã hội tụ, lộ trình phát triển tiếp theo của hệ thống hướng tới mô hình Trí tuệ nhân tạo trong Vận hành CNTT (AIOps), cụ thể bao gồm các hạng mục:

1. **Ngữ cảnh hóa tự động (Automated Contextualization):** Tích hợp tính năng tự động trích xuất và đính kèm biểu đồ tài nguyên hệ thống (CPU, RAM) tại thời điểm phát sinh sự cố vào thông báo cảnh báo.
2. **Thu thập trạng thái thời gian thực (State Snapshotting):** Xây dựng cơ chế kích hoạt tự động (Auto-trigger) để thực thi các lệnh chẩn đoán (ví dụ: `top`, `netstat`) ngay khi nhận diện rủi ro, nhằm thu thập bằng chứng kỹ thuật (forensics) tức thời.
3. **Tích hợp Mô hình Ngôn ngữ Lớn (LLMs):** Thiết lập cơ chế Webhook định tuyến cảnh báo từ Grafana tới các Tác tử AI (AI Agents). Tác tử AI đảm nhiệm phân tích tương quan đa chiều, suy luận nguyên nhân gốc rễ (Root Cause Analysis) và đề xuất Kế hoạch khắc phục tự động (Auto-remediation Plan).
4. **Kiến tạo hệ sinh thái Vòng lặp kín (Closed-loop System):** Hoàn thiện quy trình tự động hóa toàn trình: Nhận diện sự cố nhanh chóng (Grafana) → Phân tích ngữ cảnh thông minh (AI) → Phê duyệt và Thực thi khắc phục an toàn.

Tài liệu tham khảo

- [1] **Wazuh Documentation.** (2024). *Wazuh SIEM and XDR Comprehensive Documentation*. Truy cập từ: <https://documentation.wazuh.com/>
- [2] **Prometheus Authors.** (2024). *Prometheus - Monitoring system & time series database*. Truy cập từ: <https://prometheus.io/docs/>
- [3] **Grafana Labs.** (2024). *Grafana Documentation - Data visualization and Alerting*. Truy cập từ: <https://grafana.com/docs/>
- [4] **OpenSearch Contributors.** (2024). *OpenSearch Documentation - Distributed Search and Analytics Engine*. Truy cập từ: <https://opensearch.org/docs/latest/>
- [5] **MITRE Corporation.** (2024). *MITRE ATT&CK Framework - Enterprise Matrix*. Truy cập từ: <https://attack.mitre.org/>
- [6] **Samber.** (2024). *Awesome Prometheus Alerts*. Truy cập từ: <https://samber.github.io/awesome-prometheus-alerts/>